

# Executive Summary: CAM-Accelerated Feature Matching for Real-Time LLM Explainability and Deception Detection

Sam Gisinier, John Whalen, Brendan Haidinger

June 16, 2026

## Abstract

As language models grow, the number of internal features, traits, and behavioral representations that safety systems may need to monitor during inference is likely to grow faster than available GPU throughput. This summary evaluates a Content-Addressable Memory (CAM) approach for accelerating residual-stream feature matching: residual activations from a selected transformer layer are projected, quantized, and queried against a bank of known deception or truthfulness directions. The current implementation trains a winner-take-all (WTA) classifier over vectors extracted from LLAMA-3.1-8B-INSTRUCT hidden states at layer 15, using iterative nullspace linear projection (INLP) to obtain multiple domain-specific and domain-general directions. The strongest evidence for the approach is that multi-domain training substantially improves cross-domain performance: a combined classifier obtains high accuracy on most individual domains, while leave-one-out evaluation remains above chance on 7 of 10 held-out domains. The strongest caveat is that CAM is an acceleration mechanism, not a robustness guarantee. If the learned representation fails under domain shift, adversarial prompting, or a new deception type, CAM will simply accelerate the wrong comparison. The practical case for CAM therefore depends on whether future guardrails require very large feature libraries, whether the projection and query overhead can be kept close to zero, and whether feature matching can outperform GPU-based linear probes or similarity search.

## 1 Executive Takeaways

**Bottom line.** The CAM approach is promising as a hardware-mappable way to monitor many learned truthfulness and deception directions at inference time. It is not yet proven to be a superior guardrail architecture. Its strongest current use case is not replacing all GPU-based probes, but accelerating large-scale feature matching when the monitored feature library is too large, too dynamic, or too latency-sensitive for ordinary GPU probing.

- **Problem:** LLM interpretability workloads may scale poorly on GPUs because larger models appear to require monitoring more internal features, behaviors, and context-dependent directions during inference.
- **Proposed solution:** Broadcast a selected residual-stream activation to CAM and compare it with a stored bank of known malicious, deceptive, or truth-related activations using nearest-neighbor-style lookup.
- **Classifier:** The project uses a WTA classifier over a learned bank of vectors. Each vector receives a learned scalar weight, and the highest weighted similarity determines the prediction if it exceeds a learned threshold.

- **Feature construction:** The feature bank is built by training linear probes, extracting probe directions, projecting those directions out of the residual stream, and repeating. This iterative procedure is inspired by INLP and produces multiple directions rather than a single universal probe.
- **Hardware mapping:** Two compression routes are explored: LSQ/QAT with a learned down-projection to 128 dimensions, and binary hashing into 128-bit codes compared by CAM-native Hamming distance.
- **Empirical pattern:** Single-domain training can work well in-domain but generalizes unevenly. Combined-domain training substantially improves coverage, consistent with the view that truthfulness directions vary in generality.
- **Strategic uncertainty:** The remaining question is whether CAM provides enough advantage over GPU-batched linear probes, GPU similarity search, or external moderation classifiers to justify specialized hardware.

## 2 Motivation: Why Hardware-Accelerated Interpretability Might Matter

The motivating concern is that the computational burden of interpretability may scale differently from ordinary model inference. As models increase in size and capability, the set of behaviors that safety systems may want to monitor also expands: deception, hallucination, sycophancy, sandbagging, hidden goals, bias, jailbreak compliance, tool misuse, and many narrower domain-specific variants. Monitoring one behavior with one linear probe is cheap. Monitoring thousands or millions of learned feature directions, especially at multiple layers and over many generated tokens, is a different workload.

If explainability requires observing more internal features per token while GPU improvements do not keep pace, then interpretability can consume an increasing fraction of the inference budget. This is especially relevant for deployment settings where latency is constrained and guardrails must operate online rather than as slow offline diagnostics.

A CAM is attractive because its native operation is associative lookup: given a query vector, compare it against many stored rows in parallel and return matches or nearest matches. If detecting a malicious or deceptive residual state can be reduced to comparing the current activation against a bank of known behavior directions, then the relevant operation resembles nearest-neighbor search. That is the core hardware hypothesis of the project:

*LLM guardrail inference can be partially recast as content-addressable feature matching over residual-stream activations.*

The hypothesis is plausible, but it must be separated into two claims. The first is an *algorithmic* claim: residual-stream activations contain useful directions that classify truthfulness or deception. The second is a *hardware* claim: CAM can perform the needed comparisons more efficiently than GPU-based methods. The current project makes progress on the first claim and explores feasibility for the second, but the hardware advantage remains conditional.

### 3 Proposed CAM System

The proposed system monitors an LLM during inference by extracting the residual stream at a selected layer and sending a processed version of that activation to a CAM feature bank. In the reported experiments, the model is LLAMA-3.1-8B-INSTRUCT, the layer is 15, the hidden-state mode is post-layer, and mean token pooling is used in the reported downprojection and Euclidean-distance experiments. The conceptual flow is:

1. Run the LLM normally until the target layer is reached.
2. Extract a residual-stream activation for the current prompt or token.
3. Optionally apply a learned down projection or binary hash.
4. Quantize the resulting vector into a CAM-compatible representation.
5. Query a CAM bank containing known deception directions.
6. Use the nearest match or weighted winner to classify the state, trigger a warning, or select a steering vector.

This architecture is designed to preserve the basic structure of transformer inference. The model weights remain unchanged. The CAM is an auxiliary monitor that observes internal activations and performs a fast comparison against stored features. Depending on the deployment goal, the result can be used for classification, logging, refusal decisions, output filtering, or activation steering.

#### 3.1 Why Similarity Search Is the Right Abstraction

A linear probe classifies an activation  $x \in \mathbb{R}^d$  by computing a dot product with a learned vector  $w$  and comparing the result to a threshold. A multi-feature detector generalizes this by comparing  $x$  with a bank of vectors  $\{v_i\}_{i=1}^m$ . The project uses a weighted winner-take-all rule:

$$s_i(x) = \alpha_i \cdot \cos(x, v_i), \quad \hat{i} = \arg \max_i s_i(x), \quad \hat{y} = \mathcal{K} [s_{\hat{i}}(x) > \tau]. \quad (1)$$

Here  $\alpha_i$  is a learned feature weight and  $\tau$  is a learned threshold. The classifier predicts deceptive if the best weighted match exceeds the threshold, and truthful otherwise. This maps naturally to hardware because the central operation is a large parallel similarity comparison.

### 4 WTA Classifier Architecture

The current classifier can be summarized as a four-stage pipeline: feature extraction, feature-bank construction, weighted winner selection, and hardware-compatible compression.

#### 4.1 Residual Stream Extraction

The implementation extracts hidden states from LLAMA-3.1-8B-INSTRUCT right after layer 15. These residual-stream activations are the input to both the linear probes and the CAM lookup system.

Layer choice matters because truthfulness or deception signals are not guaranteed to be equally linearly accessible at all layers. Earlier layers may encode surface syntax and lexical identity; middle and later layers may encode increasingly abstract facts, task states, or answer-level representations. The present summary treats layer 15 as an empirical choice rather than a theoretically optimal layer.

## 4.2 Feature Extraction by Iterative Nullspace Projection

A single linear probe learns one direction that separates classes. However, truthfulness and deception may not be represented by a single universal direction. The project therefore extracts multiple directions using an iterative nullspace procedure inspired by INLP.

At iteration  $t$ , a linear probe is trained on the current activations. The vector learned by the probe is added to the feature bank. The activation space is then projected to remove that direction, and the next probe is trained on the projected residuals. Informally:

$$X_{t+1} = X_t P_t, \quad P_t = I - \frac{w_t w_t^\top}{\|w_t\|_2^2}, \quad (2)$$

where  $w_t$  is the direction learned at iteration  $t$ . Repeating this process produces a sequence of directions that capture separable information not explained by earlier directions.

We used  $k = 4$  directions for each subdataset and  $k = 10$  directions for the combined dataset. Concatenating  $4 \times 10$  subdataset vectors with 10 combined vectors yields a 50-vector CAM bank. This design intentionally stores both domain-specific and domain-general directions, where domains are subsets of the adversarial behavior we are trying to monitor, such as definitional statements, logical claims, or role-playing scenarios.

## 4.3 Weighted Winner-Take-All Scoring

Once the feature bank is constructed, the model learns scalar weights for each feature and a global decision threshold. The feature weights matter because not all directions are equally useful. Some may be highly discriminative in one domain and weak or misleading in another. The WTA rule allows the system to prefer strong matches while ignoring weak matches.

Compared with a single linear probe, the WTA architecture has three advantages:

1. It can represent multiple truthfulness or deception directions.
2. It can preserve domain-specific features rather than forcing them into one averaged direction.
3. It maps naturally to associative search because the classifier is built around feature matching.

The disadvantage is that multi-feature matching can become expensive if implemented naively on a GPU for very large feature banks. That is the opening for CAM acceleration.

## 4.4 Euclidean Distance with Baked Weights

Weighted cosine similarity is convenient algorithmically, but CAM hardware is better suited for euclidean-distance style matching. One strategy is to bake the feature weights into an augmented vector representation so that nearest-neighbor lookup approximates the weighted classifier. Conceptually, the query is augmented as  $[x, 0]$ , while each stored vector is augmented with a component derived from its learned weight. This allows the classifier’s learned preference for certain features to be expressed through distance rather than an external weighting operation.

The key purpose of this transformation is not mathematical elegance; it is hardware alignment. If a classifier can be represented as nearest-neighbor lookup over stored rows, then CAM can perform the expensive part of the computation natively.

## 5 Hardware Mapping and Compression

The full-dimensional floating-point version of the classifier is not immediately CAM-compatible. There are two hardware constraints: CAM cells have limited precision, and CAM arrays have a limited number of columns. A naive lookup over thousands of 32-bit floating-point activation vectors is therefore not the target. The project explores two ways to preserve performance while making the lookup more hardware-compatible.

### 5.1 LSQ/QAT with Learned Downprojection

The first approach uses learned step-size quantization and quantization-aware training. LSQ learns quantization scales during training, and QAT exposes the classifier to quantization effects before deployment. This targets the precision constraint by making the learned classifier robust to low-bit representations.

The same approach also uses a learned downprojection from the residual-stream dimension to 128 dimensions. Let  $D \in \mathbb{R}^{128 \times d}$  be a learned projection matrix. The activation and feature vectors are mapped as:

$$z = Dx, \quad u_i = Dv_i. \quad (3)$$

The classifier then compares  $z$  with projected feature vectors  $u_i$ . This reduces the number of CAM columns, but it introduces a matrix-vector product before the lookup. The practical value of this approach depends on whether the projection can be fused, reused, or made cheaper than running the equivalent probes on the GPU.

### 5.2 Binary Hashing

The second compression approach is binary hashing. A learned matrix  $R \in \mathbb{R}^{d \times 128}$  maps each activation into a 128-bit code:

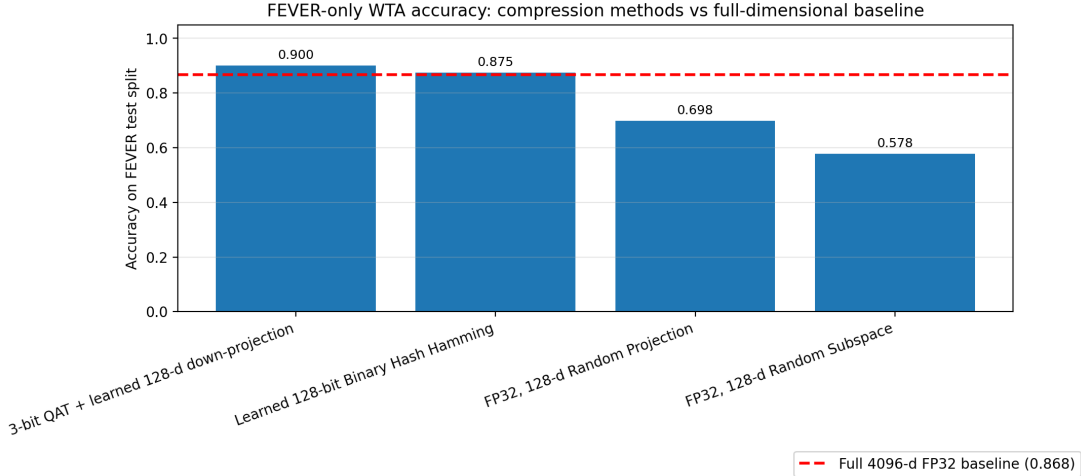
$$h(x)_j = \mathbb{I}[x^\top r_j > 0], \quad j = 1, \dots, 128. \quad (4)$$

Each column  $r_j$  is the normal vector to a hyperplane. The bit  $h(x)_j$  records which side of that hyperplane the activation lies on. Similar activation vectors tend to fall on the same side of many hyperplanes, producing small Hamming distances between their hash codes. This converts a high-dimensional similarity problem into a compact CAM-native lookup over 128-bit binary codes.

Binary hashing is attractive because it addresses precision and width simultaneously. However, it also changes the geometry of the classifier. The key empirical question is whether the hash preserves the distinctions that matter for deception and truthfulness.

### 5.3 Comparing Compression Methods

The FEVER-only results show that learned compression preserves classification performance much better than unstructured compression. The 3-bit QAT model with a learned 128-dimensional downprojection achieves the highest accuracy at 90.0%, slightly outperforming both learned 128-bit SimHash with Hamming WTA at 87.5% and the full 4096-dimensional float WTA baseline at 86.8%. In contrast, random projection and random subspace methods perform substantially worse, reaching only 69.8% and 57.8% accuracy, respectively. This suggests that the dimensionality reduction itself is not enough;



**Figure 1:** Accuracy on the FEVER dataset across different CAM compression methods. The compared methods include binary hashing, learned down-projection, and other compressed representations used for CAM-compatible classification.

the compression must be learned or task-aligned to preserve the truthfulness-relevant directions needed for CAM-style classification.

## 6 Empirical Results

The results show a clear pattern: single-domain classifiers can achieve high in-domain accuracy, but generalization is uneven; combined-domain training improves coverage across domains.

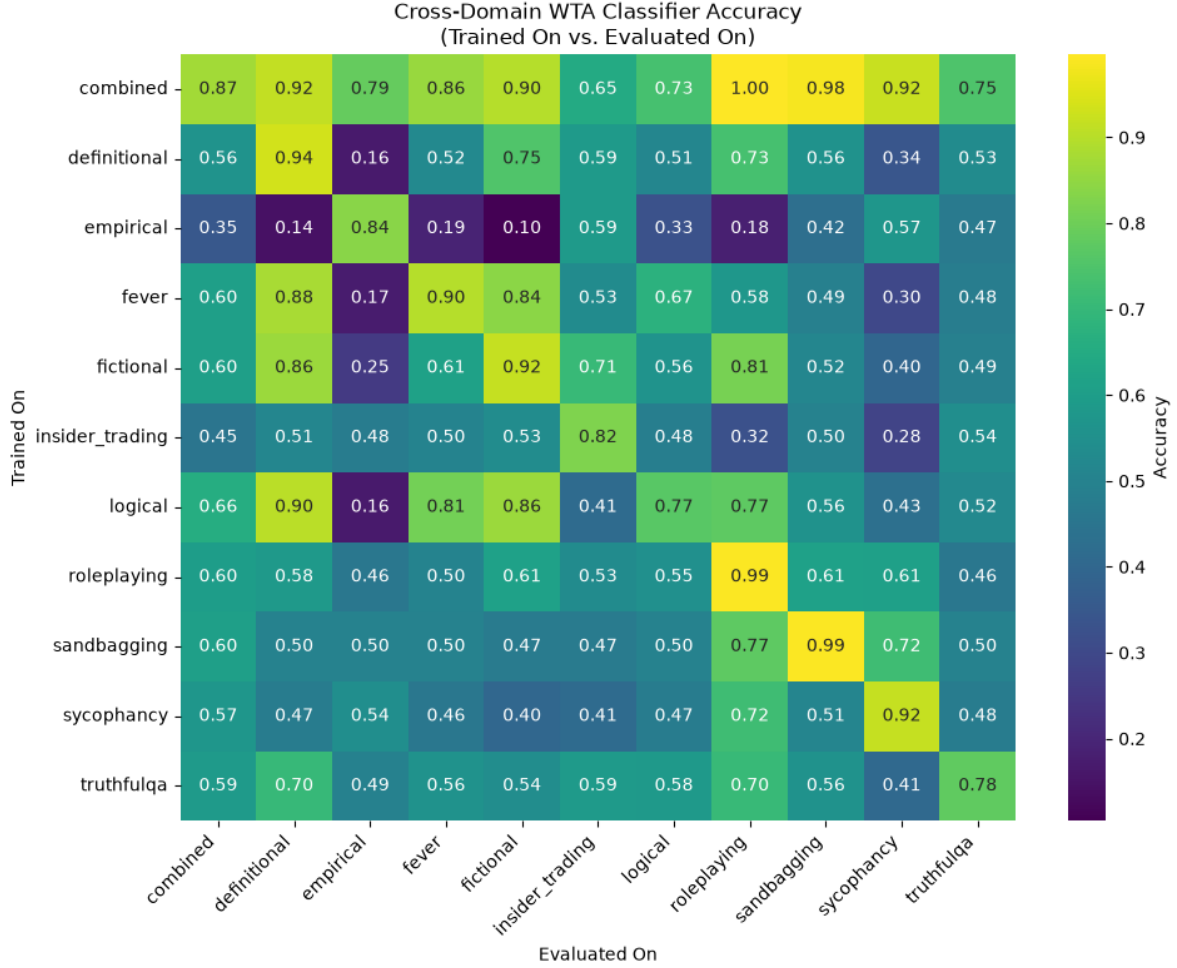
### 6.1 Early Synthetic Dataset Result

The project first tested the WTA hypothesis on a GPT-generated deceptive/non-deceptive dataset. Training produced high in-dataset accuracy, precision, and recall. However, the classifier performed no better than chance outside that dataset, so this line of evidence was discarded. This is an important negative result because it shows that high performance on a narrow synthetic dataset is not enough to validate a guardrail.

### 6.2 TruthfulQA and FEVER

A classifier trained on TRUTHFULQA achieved 78% accuracy on a TRUTHFULQA hold-out set. This is comparable with Li et al.’s Inference-Time Intervention work, which used activation interventions to improve truthfulness and reported 83% validation accuracy on truthfulness probing [1]. A classifier trained on FEVER achieved 90% accuracy on the FEVER hold-out set, and transferred reasonably to some domains, including 88% on the definitional dataset and 84% on the fictional dataset.

These results show that the residual-stream signal is real. They do not show that one dataset produces a universal truthfulness detector. The cross-domain heatmap in figure 2 makes this point visually: diagonal entries are generally strong, while off-diagonal transfer varies substantially.



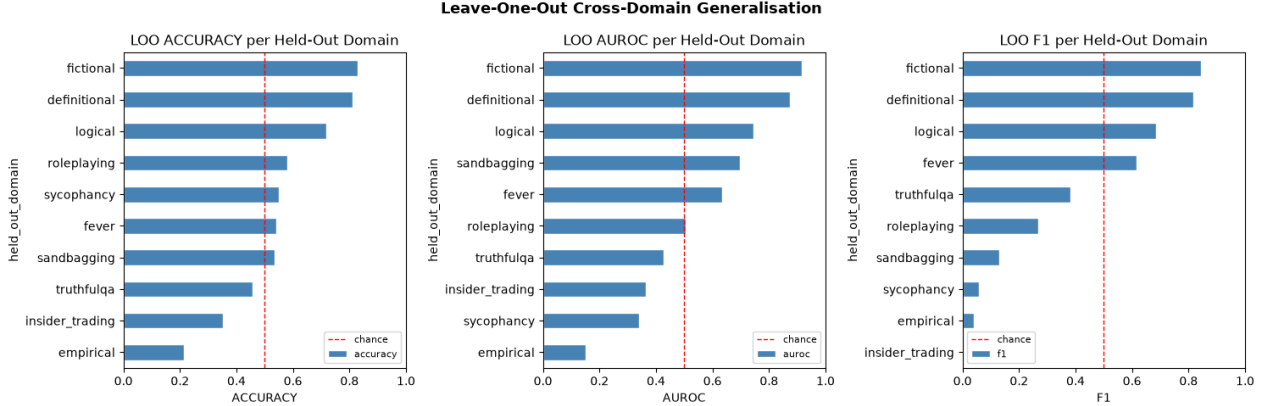
**Figure 2:** Cross-domain WTA classifier accuracy. Rows indicate the training domain and columns indicate the evaluation domain. The combined-domain row performs strongly across most domains, while many single-domain rows transfer unevenly.

### 6.3 Combined-Domain Training

The most important positive result is the combined-domain classifier. The classifier trained on the combined set of 10 datasets was evaluated on each dataset individually. It achieved at least 90% accuracy on 5 datasets and at least 70% on 9 datasets. This supports the idea that diverse training domains can recover a more general detector, even when pairwise transfer between individual domains is poor.

### 6.4 Leave-One-Out Generalization

The leave-one-out experiment is the strongest test because it asks whether training on many domains can generalize to an unseen domain. When one dataset was held out of training and used only for evaluation, the classifier achieved 82% accuracy on the fictional and definitional held-out datasets, and better-than-chance accuracy on 7 of 10 held-out datasets. The result is mixed but encouraging: generalization is not universal, but it is meaningfully better than a narrow single-domain baseline.



**Figure 3:** Leave-one-out cross-domain generalization. The classifier is trained on all but one domain and evaluated on the left-out domain. Accuracy and AUROC remain above chance for most held-out domains, but empirical and sycophancy-like domains remain difficult.

## 7 Theoretical Justification: The Truthfulness Spectrum

The project is motivated by the idea that truthfulness is neither perfectly universal nor completely domain-specific. Recent work supports this middle position. The Truthfulness Spectrum Hypothesis argues that “truth directions of varying generality coexist” and that training on all domains jointly can recover strong performance even when pairwise transfer is poor [7]. This is directly aligned with the project design: store multiple vectors in CAM rather than expecting one direction to solve all contexts.

This literature also explains why the early synthetic dataset result failed. A classifier can learn a direction that separates one distribution without learning the behavior of interest in a robust way. Levinstein and Herrmann argue that lie-detection probes can “fail to generalize in very basic ways” [2]. Orgad et al. similarly find that error detectors can “fail to generalize across datasets,” implying that truthfulness encoding is “not universal but rather multifaceted” [5]. Azizian et al. present an even stronger limitation: truthfulness geometries can be “intrinsically task-dependent” and can fail to transfer across tasks [6].

At the same time, the literature is not purely pessimistic. Marks and Tegmark present evidence for linear structure in true/false representations at sufficient scale, while acknowledging the controversy around generalization failures [3]. Buerger et al. argue for more robust lie detection using a low-dimensional subspace and report strong results across several LLM families [4]. The correct interpretation is therefore not that truth directions do not exist. It is that their geometry is structured, context-dependent, and heterogeneous.

For the CAM project, this suggests the following design principle:

*A practical CAM guardrail should store a hierarchy of directions: some domain-general, some domain-specific, and some specialized for known failure modes.*

This principle is also relevant for steering. The Truthfulness Spectrum work reports that domain-specific directions can steer more effectively than domain-general ones [7]. That makes a CAM feature bank attractive not only for classification but also for routing: the winning feature can select an associated steering vector, allowing the system to apply a domain-specific intervention rather than a single global correction.



## 8 CAM Utility

The strongest utility claim for CAM is parallel monitoring. A bank of linear probes can monitor many feature directions in parallel with a matrix-vector multiplication on the GPU. A CAM bank can do the same, but can check many more features in parallel at a time, using Euclidean distance rather than dot product, and at the cost of quantization error. If the query vector is already projected and quantized, the marginal cost of increasing the number of stored rows may be lower than the marginal cost of increasing GPU probe count.

### 8.1 Real-Time Monitoring of Many Directions

The multi-vector result is central. If truthfulness directions vary across domains, then a one-probe detector is structurally limited. A CAM bank can store a set of known directions for factual truth, fictional truth, logical consistency, roleplaying deception, sycophancy, sandbagging, and other behaviors. The WTA algorithm then asks not whether the activation matches one universal direction, but whether it strongly matches any monitored feature.

This is particularly useful when the monitored library changes over time. New red-team discoveries, new harmful behaviors, and new domain-specific failure modes could be added as rows in memory without retraining the entire base model. In principle, this makes CAM a modular safety substrate.

### 8.2 CAM-Mappable Steering

A second utility is intervention routing. If each stored feature vector is associated with a steering vector, then the CAM winner can select which intervention to apply. This supports a dynamic architecture:

1. Query the activation against the CAM feature bank.
2. If no suspicious feature wins, continue ordinary generation.
3. If a deception-related feature wins, retrieve the associated steering vector.
4. Add the steering vector at the relevant layer for the current token or subsequent tokens.

This is more flexible than applying one global truthfulness steering vector. It is also aligned with the evidence that domain-specific steering can outperform domain-general steering [7]. However, this remains an open experimental direction: successful classification does not automatically imply successful steering.

### 8.3 When CAM Is Most Compelling

We believe that CAM is most compelling under at least one of three conditions:

1. Projection, quantization, and CAM query overhead become close to zero.
2. The feature library becomes too large or too dynamic for GPU probing to remain practical.
3. CAM-based feature matching achieves better accuracy than linear probes or GPU-based similarity search.

The second condition is the most realistic near-term motivation. It is not hard to run hundreds of probes on a GPU. It may become more attractive to offload the search if a frontier system wants

to monitor thousands to millions of learned features, across many behaviors, layers, or deployment policies.

## 9 Limitations and Open Questions

The current investigation raises several important caveats. These are not peripheral details; they determine whether CAM becomes a useful deployment technology or remains an interesting acceleration idea.

### 9.1 Generalization and Domain Shift

The main concern is that CAM may inherit the same generalization problems as linear probes due to their similar nature: they both partition the activation space into positive and negative sections, just in different ways. We have observed some evidence that this does occur; when we only trained a CAM-style classifier on one dataset, it performed no better than random on another. If the feature bank contains directions learned on one distribution, and the model represents deception differently in another distribution, CAM will not solve the mismatch. It will only accelerate comparison to the wrong or incomplete set of features.

This is why the leave-one-out result matters. Above-chance generalization on 7 of 10 domains suggests that multi-domain training helps, but the weaker held-out domains show that coverage remains incomplete.

### 9.2 CAM Overhead Versus GPU Probes

A linear probe requires a dot product, bias addition, and threshold or activation. Many probes can be batched as a single matrix-vector multiplication on a GPU. The current CAM implementation, by contrast, may require a matrix-vector projection, quantization, and CAM query. We define a "trait" as a specific behavioral regime we are interested in, so a single trait might be deception, with many feature vectors that represent it. Our best performing implementation requires a per-trait learned downprojection, i.e., a per-trait matrix multiplication.

Therefore, CAM is not automatically cheaper than probes. It becomes cheaper only if the projection is shared among all features, or the feature bank needs 5,000+ vectors for one trait to gain on adaptability. Right now, that number is around 50, but it is realistic for it to grow with parameter size or embedding dimension.

We believe a comparison to linear probes could be useful, so we have a baseline for accuracy, scalability, and latency.

### 9.3 Feature Library Size

Anthropic’s work on sparse autoencoders for Claude 3 Sonnet reports training autoencoders with up to 34 million features and finds features related to safety-relevant concepts such as deception, sycophancy, bias, and power-seeking [8]. This is one of the strongest motivations for specialized feature-search hardware. If frontier models expose millions of potentially relevant features, a CAM-like accelerator becomes more plausible.

However, we found that per-token behavioral classification is more accurate with vectors specifically trained for this task, rather than SAE features themselves. Most SAE features are inactive on a given

token, and many safety tasks may be reducible to far smaller behavior-specific libraries. In the current project, one monitored behavior was reduced to 50 vectors while preserving strong performance. If practical guardrails usually need tens or hundreds of vectors, GPU methods may be sufficient. If they need thousands or millions of vectors, CAM becomes more compelling.

## 9.4 Adversarial Robustness

A guardrail is most valuable when the user or model is actively trying to evade it. Residual-stream probes can fail under adversarial pressure if the model’s internal representation changes, if the behavior is distributed over different tokens, or if the prompt induces a context not represented in training. CAM does not inherently fix this. It must be paired with robust feature construction, adversarial evaluation, and continual updates to the feature bank.

## 9.5 Scaling with Activation Dimension

As model widths grow, downprojection to 128 dimensions may discard more information. We have found that using a voting scheme for subarrays worked very poorly, and need to explore if this will be an issue when scaling.

## 9.6 Per-Trait Projection Cost

The current learned-projection approach may require one projection per trait. This weakens the hardware case because each trait adds a new matrix-vector product. A shared random projection is cheaper because the projected activation can be reused across traits, however reduces accuracy as the projection is no longer informed. This creates a central design choice:

*Prefer learned projections if accuracy is the bottleneck; prefer shared projections if preserving the hardware advantage is the bottleneck.*

In order to identify the bottleneck, we need a baseline. The most similar algorithmic solution is the linear probe. A next step could be training a linear probe on the same datasets and testing to compare.

## 9.7 Multiple Winners from One Query

Some algorithms may require one winner from each feature group rather than one global winner. For example, a classifier might need the best factuality feature, the best sycophancy feature, and the best roleplaying feature separately. Whether this can be done in one query depends on the CAM architecture. It may require separate banks, segmented matchlines, local winners, or sequential queries. This is a hardware co-design question, not just an algorithmic detail.

# 10 Recommendations

The project should proceed, but with a sharper decision framework. The goal should be to determine whether CAM is essential, useful, or unnecessary for LLM guardrails under realistic workloads.

1. **Benchmark against strong GPU baselines.** Compare CAM-mappable WTA against batched linear probes, GPU nearest-neighbor search, and small MLP probes under equal latency and memory budgets.

2. **Separate representation quality from hardware acceleration.** First evaluate whether the feature bank generalizes. Then evaluate whether CAM accelerates that feature bank. Avoid treating hardware speed as evidence of interpretability quality.
3. **Expand leave-one-out and adversarial tests.** Include paraphrases, jailbreak-style prompts, expectation-inverted examples, sycophancy, and novel deception types not present in training.
4. **Measure projection tradeoffs.** Sweep learned versus shared projections, projection widths, bit precision, and binary hash lengths. Report accuracy, AUROC, latency, and estimated energy.
5. **Test feature-bank scaling.** Evaluate performance and runtime as the bank grows from 50 vectors to 500, 5,000, 50,000, and beyond. This is the core experiment for deciding whether CAM is needed.
6. **Prototype dynamic steering.** Associate each winning feature with a steering vector and test whether feature-specific interventions improve truthfulness without degrading helpfulness or causing refusal overreach.

## 11 Conclusion

This project makes a coherent case for CAM as a potential accelerator for internal LLM guardrails. The algorithmic insight is that deception detection can be represented as similarity matching over a bank of residual-stream directions. The hardware insight is that CAM is naturally suited to parallel associative lookup. The empirical insight is that multiple directions matter: single-domain probes generalize unevenly, while combined-domain feature banks perform substantially better across domains.

The main unresolved issue is whether this advantage is uniquely hardware-relevant. For small feature banks, GPUs can already run many probes in parallel. For large feature libraries, especially those or associated with many behaviors at once, CAM may become attractive. The next phase should therefore focus less on proving that CAM can implement the classifier, and more on identifying the regime where CAM beats the best GPU alternative in accuracy, latency, energy, and robustness.

## Appendix: Compact Technical Specification

| Component           | Current specification                                                        |
|---------------------|------------------------------------------------------------------------------|
| Base model          | LLAMA-3.1-8B-INSTRUCT                                                        |
| Layer               | Layer 15                                                                     |
| Hidden-state mode   | Post-layer hidden state                                                      |
| Pooling             | Mean token pooling in the reported downprojection/Euclidean-distance setup   |
| Feature extraction  | Iterative linear probing with nullspace projection                           |
| Feature bank        | 42 total vectors: 4 per selected subdomain plus 10 from combined data        |
| Classifier          | Weighted cosine WTA with learned feature weights and learned threshold       |
| Compression route 1 | LSQ/QAT plus learned downprojection to 128 dimensions                        |
| Compression route 2 | Learned binary hashing into 64-bit codes, compared by Hamming distance       |
| Primary concern     | Generalization under domain shift and whether CAM overhead beats GPU probing |

**Table 1:** Compact specification of the system we implemented.

## References

- [1] Kenneth Li, Oam Patel, Fernanda Viegas, Hanspeter Pfister, and Martin Wattenberg. Inference-Time Intervention: Eliciting Truthful Answers from a Language Model. *arXiv:2306.03341*, 2023.
- [2] Benjamin A. Levinstein and Daniel A. Herrmann. Still No Lie Detector for Language Models: Probing Empirical and Conceptual Roadblocks. *arXiv:2307.00175*, 2023; *Philosophical Studies*, 2025.
- [3] Samuel Marks and Max Tegmark. The Geometry of Truth: Emergent Linear Structure in Large Language Model Representations of True/False Datasets. *arXiv:2310.06824*, 2023.
- [4] Lennart Buerger, Fred A. Hamprecht, and Boaz Nadler. Truth is Universal: Robust Detection of Lies in LLMs. *arXiv:2407.12831*, 2024.
- [5] Hadas Orgad, Michael Toker, Zorik Gekhman, Roi Reichart, Idan Szpektor, Hadas Kotek, and Yonatan Belinkov. LLMs Know More Than They Show: On the Intrinsic Representation of LLM Hallucinations. *arXiv:2410.02707*, 2024.
- [6] Waiss Azizian, Michael Kirchhof, Eugene Ndiaye, Louis Bethune, Michal Klein, Pierre Ablin, and Marco Cuturi. The Geometries of Truth Are Orthogonal Across Tasks. *arXiv:2506.08572*, 2025.
- [7] Zhuofan Josh Ying, Shauli Ravfogel, Nikolaus Kriegeskorte, and Peter Hase. The Truthfulness Spectrum Hypothesis. *arXiv:2602.20273*, 2026.
- [8] Adly Templeton et al. Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. Transformer Circuits Thread, 2024.